Первая система с открытым ключом — система Диффи–Хеллмана

Эта криптосистема была открыта в середине 70-х годов американскими учеными:

- **Уитфилдом Диффи (Whitfield Diffie),**
- **Мартином Хеллманом (Martin Hellman)**.

Она привела к настоящей революции в криптографии и ее практических применениях.



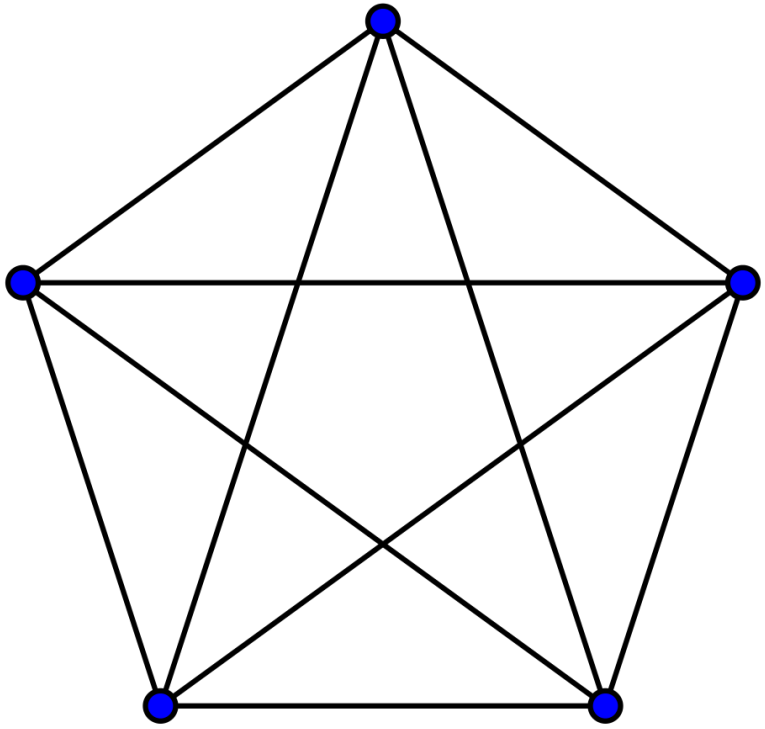
Это первая система, которая позволяла защищать информацию без использования секретных ключей, передаваемых по защищенным каналам.

На рисунке слева направо:

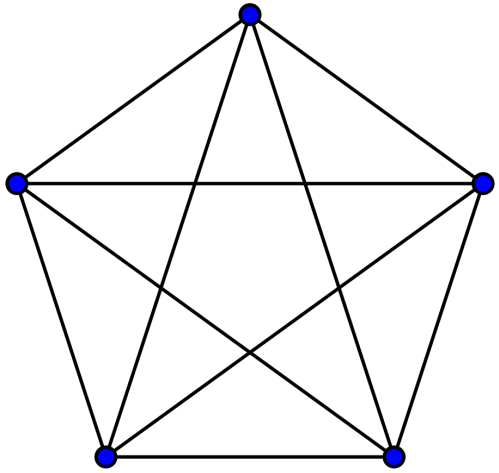
Мартин Хеллман, Уитфилд Диффи

Рассмотрим сеть связи с N пользователями, где N – большое число. Пусть мы хотим организовать секретную связь для каждой пары из них.

Тогда потребуется столько ключей: $C_N^2 = \frac{N(N-1)}{2} \approx \frac{N^2}{2}$



Если мы будем использовать обычную систему распределения секретных ключей, то каждая пара абонентов должна быть снабжена своим секретным ключом, т.е. всего ключей:

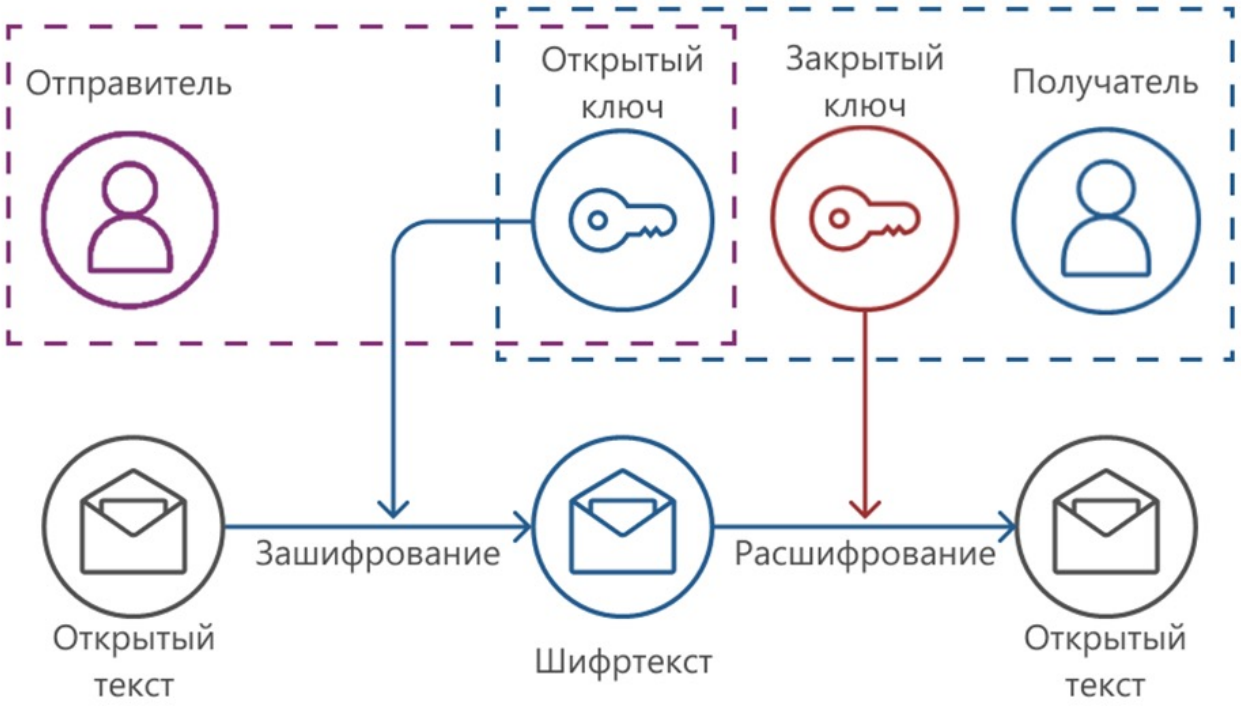


$$C_N^2 = \frac{N(N-1)}{2} \approx \frac{N^2}{2}$$

При большом числе абонентов система снабжения их секретными ключами становится очень громоздкой и дорогостоящей.

У. Диффи и М. Хеллман решили эту проблему за счет открытого распространения и вычисления ключей.

Впервые появилось асимметричное шифрование!



У каждого абонента есть своя открытая и закрытая (секретная) информация.

Схема Диффи-Хеллмана

Строится система для нескольких абонентов: A, B, C.

Общие открытые параметры системы:

p – большое простое число,

g – первообразный корень по модулю p, $1 < g < p - 1$

Секретные параметры абонентов:

X_A, X_B, X_C – большие случайные числа в диапазоне $[2; p - 1]$

Открытые параметры абонентов:

$$Y_A = g^{X_A} \bmod p,$$

$$Y_B = g^{X_B} \bmod p,$$

$$Y_C = g^{X_C} \bmod p.$$

Таблица 2.2. **Ключи пользователей в системе Диффи–Хеллмана**

Абонент	Секретный ключ	Открытый ключ
A	X_A	Y_A
B	X_B	Y_B
C	X_C	Y_C

Допустим, абонент A решил организовать сеанс связи с B, при этом обоим абонентам доступна открытая информация из табл. 2.2. Абонент A сообщает B по открытому каналу, что он хочет передать ему сообщение. Затем абонент A вычисляет величину

$$Z_{AB} = (Y_B)^{X_A} \bmod p \tag{2.10}$$

(никто другой кроме A этого сделать не может, так как число X_A секретно). В свою очередь, абонент B вычисляет число

$$Z_{BA} = (Y_A)^{X_B} \bmod p. \tag{2.11}$$

Первообразный корень по модулю **m** — целое число **g** такое, что

$$g^{\varphi(m)} \equiv 1 \pmod{m}$$

и

$$g^{\ell} \not\equiv 1 \pmod{m} \text{ при } 1 \leq \ell < \varphi(m),$$

где $\varphi(m)$ — функция Эйлера.

Чтобы не проверять все ℓ от 1 до $\varphi(m)$, достаточно проверить три условия:

1. Является ли g числом взаимно простым с m, и если нет - то это не первообразный корень.
2. Так как $\varphi(m)$ - всегда чётное число для всех $m > 2$, то $\varphi(m)$ имеет как минимум один простой делитель - простое число 2, следовательно, для того, чтобы отсеять значительное количество не-корней, достаточно проверить $g^{\varphi(m)/2} \not\equiv 1 \pmod{m}$ для числа, подходящего на первообразный корень по модулю m.[1] Если результат +1 $\equiv$ m , то g - не корень, в ином случае результат -1 $\equiv$ m, когда g - это возможно первообразный корень.
3. Далее следует убедиться, что $g^{\ell} \not\equiv 1 \pmod{m}$ для всех $\ell = \frac{\varphi(m)}{p}$, где p - простой делитель числа $\varphi(m)$, полученный в результате его факторизации.

В практических реализациях для X_A и X_B используются числа порядка 10^{100} и p порядка 10^{300}. Число **g** не обязано быть большим и обычно имеет значение в пределах первого десятка.

Другими словами, **первообразный корень** — это образующий элемент мультипликативной группы кольца вычетов по модулю **m**.

Сравнение двух целых чисел по модулю натурального числа m — математическая операция, позволяющая ответить на вопрос о том, дают ли два выбранных целых числа при делении на **m** один и тот же остаток.

Любое целое число при делении на **m** дает один из **m** возможных остатков: число от 0 до **m – 1**.

Это значит, что все целые числа можно разделить на **m** групп, каждая из которых отвечает определённому остатку от деления на **m**.

Эти группы называются **классами вычетов по модулю m**, а содержащиеся в них целые числа — **вычетами** по модулю **m**.

Арифметические операции с остатками чисел по фиксированному модулю образуют **модульную арифметику**, которая широко применяется в **математике**, **информатике** и **криптографии**.

Первообразный корень по модулю **m** — целое число **g** такое, что

$$g^{\varphi(m)} \equiv 1 \pmod{m}$$

и

$$g^{\ell} \not\equiv 1 \pmod{m} \text{ при } 1 \leq \ell < \varphi(m),$$

где $\varphi(m)$ — функция Эйлера.

Число 3 является первообразным корнем по модулю 7. Чтобы в этом убедиться, достаточно каждое число от 1 до 6 представить как некоторую степень тройки по модулю 7:

$$\begin{aligned}
3^1 &\equiv 3 \pmod{7} \\
3^2 &\equiv 2 \pmod{7} \\
3^3 &\equiv 6 \pmod{7} \\
3^4 &\equiv 4 \pmod{7} \\
3^5 &\equiv 5 \pmod{7} \\
3^6 &\equiv 1 \pmod{7}
\end{aligned}$$

Задача быстрого нахождения чисел *p* и *g*

При произвольно заданном ***p*** она может оказаться трудной задачей, связанной с разложением на простые множители числа ***p-1***.

Для обеспечения высокой стойкости рассмотренной системы число ***p-1*** должно обязательно содержать большой простой множитель (в противном случае можно быстро вычислить дискретный логарифм с помощью алгоритма Полига–Хеллмана).

Поэтому часто рекомендуют использовать следующий подход. Простое число p выбирается таким, чтобы выполнялось равенство

$$p = 2q + 1,$$

где q — также простое число. Тогда в качестве g можно взять любое число, для которого справедливы неравенства

$$1 < g < p - 1 \quad \text{и} \quad g^q \bmod p \neq 1.$$

При работе алгоритма каждая сторона:

1. Генерирует случайное натуральное число — закрытый ключ

2. Совместно с удалённой стороной устанавливает открытые параметры **p** и **g** (обычно значения **p** и **g** генерируются на одной стороне и передаются другой), где **p** является случайным простым числом,

а **q = (p-1)/2** также должно быть случайным простым числом (для повышения безопасности).

g является первообразным корнем по модулю p (также является простым числом).

3. Вычисляет открытый ключ, используя преобразование над закрытым ключом

$$Y_A = g^{X_A} \bmod p$$

$$Y_B = g^{X_B} \bmod p$$

4. Обменивается открытыми ключами с удалённой стороной

5. Вычисляет общий секретный ключ **Z**, используя открытый ключ удаленной стороны B и свой закрытый ключ.

На стороне A:

$$Z_{AB} = Y_B{}^{X_A} \ mod \ p =$$

$$= (g^{X_B} \ mod \ p)^{X_A} \ mod \ p =$$

$$= (g^{X_B})^{X_A} \ mod \ p =$$

$$= g^{X_B X_A} \ mod \ p$$

На стороне B:

$$Z_{BA} = Y_A{}^{X_B} \ mod \ p =$$

$$= (g^{X_A} \ mod \ p)^{X_B} \ mod \ p =$$

$$= (g^{X_A})^{X_B} \ mod \ p =$$

$$= g^{X_A X_B} \ mod \ p$$

Утверждение 2.2. $Z_{AB} = Z_{BA}$.

Доказательство. Действительно,

$$Z_{AB} = (Y_B)^{X_A} \bmod p = (g^{X_B})^{X_A} \bmod p = \\ = g^{X_A X_B} \bmod p = (Y_A)^{X_B} \bmod p = Z_{BA}.$$

Отметим главные свойства системы:

1) абоненты A и B получили одно и то же число $Z = Z_{AB} = Z_{BA}$, которое не передавалось по открытой линии связи;

2) Ева не знает секретных чисел $X_A, X_B, \ldots$, поэтому не может вычислить число Z_{AB} (вообще говоря, она могла бы попытаться найти секретное число X_A по Y_A (см. (2.9)), однако при больших p это практически невозможно (требуются миллионы лет)).

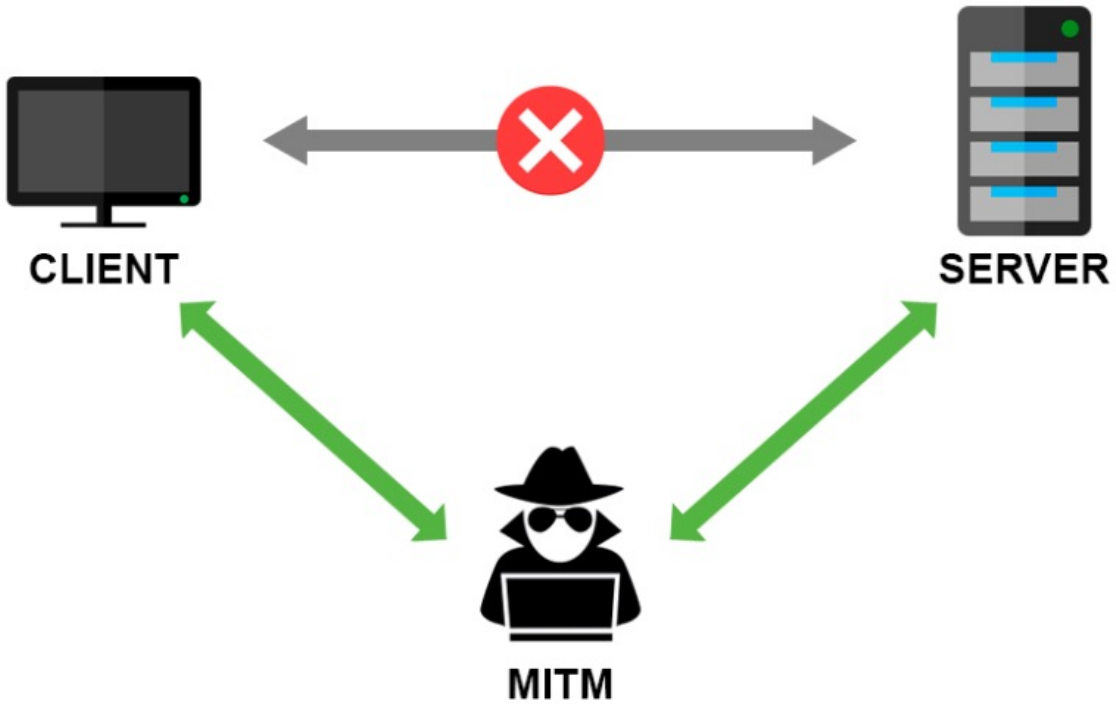
Пример 2.2. Пусть $p = 23 = 2 \cdot 11 + 1$ $(q = 11)$. Выберем параметр g. Попробуем взять $g = 3$. Проверим: $3^{11} \bmod 23 = 1$ и значит, такое g не подходит. Возьмем $g = 5$. Проверим: $5^{11} \bmod 23 = 22$. Итак, мы выбрали параметры $p = 23$, $g = 5$. Теперь каждый абонент выбирает секретное число и вычисляет соответствующее ему открытое число. Пусть выбраны $X_A = 7$, $X_B = 13$. Вычисляем

$Y_A = 5^7 \bmod 23 = 17$, $Y_B = 5^{13} \bmod 23 = 21$. Пусть A и B решили сформировать общий секретный ключ. Для этого A вычисляет $Z_{AB} = 21^7 \bmod 23 = 10$, а B вычисляет $Z_{BA} = 17^{13} \bmod 23 = 10$. Теперь они имеют общий ключ 10, который не передавался по каналу связи.

Атака посредника - Man in the middle (MITM)

Злоумышленник тайно ретранслирует и при необходимости изменяет связь между двумя сторонами, которые считают, что они непосредственно общаются друг с другом.

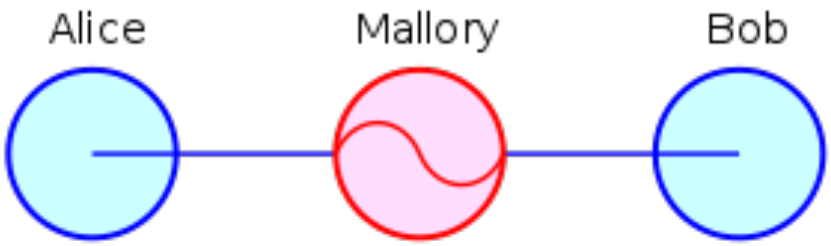
Взломщик осуществляет вмешательство в протокол передачи, **удаляя или искажая информацию**.



Данная атака направлена на обход взаимной аутентификации или отсутствие таковой.

Она будет успешной только тогда, когда злоумышленник имеет возможности:

- выдать себя за каждую конечную точку
- или оставаться незамеченным в качестве промежуточного узла.



Пример атаки на алгоритмическом языке

1. Алиса отправляет Бобу сообщение, которое перехватывает Мэлори:

   Алиса *«Привет, Боб, это Алиса. Пришли мне свой открытый ключ.»* → Мэлори
   Боб

2. Мэлори пересылает сообщение Бобу; Боб не может догадаться, что это сообщение не от Алисы:

   Алиса Мэлори *«Привет, Боб, это Алиса. Пришли мне свой открытый ключ.»* →
   Боб

3. Боб посылает свой ключ:

   Алиса Мэлори ← *[ключ Боба]* Боб

4. Мэлори подменяет ключ Боба своим и пересылает сообщение Алисе:

   Алиса ← *[ключ Мэлори]* Мэлори Боб

5. Алиса шифрует сообщение ключом Мэлори, считая, что это ключ Боба, и только он может расшифровать его:

   Алиса *«Встречаемся на автобусной остановке!» [зашифровано ключом Мэлори]* → Мэлори Боб

6. Мэлори расшифровывает сообщение, читает его, модифицирует его, шифрует ключом Боба и отправляет его:

   Алиса Мэлори *«Жди меня у входа в музей в 18:00.» [зашифровано ключом Боба]* → Боб

7. Боб считает, что это сообщение Алисы.

Атака на протокол Диффи-Хеллмана

Рассмотрим атаку на протокол выработки общего секрета Диффи-Хеллмана между сторонами ***A*** и ***B***. Допустим, криптоаналитик ***E*** имеет возможность не только перехватывать сообщения, но и подменять их своими, то есть осуществлять активную атаку:

$$A \leftrightsquigarrow E \leftrightsquigarrow B.$$

Перехват и подмена ключей

1. Сторона ***A*** отправляет сообщение стороне ***B***:

$$A \overset{E}{\rightsquigarrow} B: \ g^x \mod p.$$

2. Криптоаналитик ***E*** перехватывает сообщение стороны ***A*** и подменяет его, отправляя стороне ***B*** уже другое сообщение :

$$E \to B: \ g^z \mod p.$$

3. Сторона ***B*** отправляет сообщение стороне ***A***:

$$A \overset{E}{\leftsquigarrow} B: \ g^y \mod p.$$

4. Криптоаналитик ***E*** перехватывает сообщение стороны ***B*** и подменяет его, отправляя стороне ***A*** какое-то своё сообщение :

$$A \leftarrow E: \ g^z \mod p.$$

5. Результатом данных действий является образование двух каналов связи криптоаналитика ***E*** со сторонами ***A*** и ***B***, причем сторона ***A*** считает что общается со стороной ***B*** при помощи секретного ключа K_{AE}, а сторона ***B*** отправляет сообщения при помощи ключа K_{BE}. При этом стороны ***A*** и ***B*** не подозревают, что обмен сообщениями происходит не напрямую, а через криптоаналитика ***E***:

$$K_{AE} = g^{xz} \mod p,$$
$$K_{BE} = g^{yz} \mod p.$$

Подмена сообщений

1. Сторона ***A*** отправляет сообщение m стороне ***B***, зашифрованное при помощи ключа K_{AE}:

$$A \overset{E}{\nrightarrow} B : E_{K_{AE}}(m).$$

2. Криптоаналитик ***E*** перехватывает это сообщение, расшифровывает ключом K_{AE}, при необходимости, изменяет его на m', зашифровывает ключом K_{BE} и отправляет стороне ***B***: K_{AE}:

$$E \to B : E_{K_{BE}}(m').$$

3. Аналогичные действия криптоаналитик ***E*** предпринимает при передачи сообщений от ***B*** к ***A***.

Сценарии атаки

- **Атака на системы с открытым ключом**
- **Внедрение вредоносного кода**
- **Downgrade Attack**
- **Публичные средства коммуникации**

Обнаружение MITM-атаки

- **IP-адрес сервера**
- **DNS-сервер**
- **X.509-сертификат сервера**
  - Подписан ли сертификат самостоятельно?
  - Подписан ли сертификат центром сертификации?
  - Был ли сертификат аннулирован?
  - Менялся ли сертификат недавно?
  - Получали ли другие клиенты в интернете такой же сертификат?

Обобщенный алгоритм Евклида

Обычный алгоритм Евклида находит НОД (наибольший общий делитель) двух чисел.

Расширенный алгоритм Евклида идёт дальше: **он находит** не только **НОД (a,b)**, но и такие целые числа **x** и **y**, что выполняется соотношение Безу (2.13):

$$ax + by = \gcd(a, b). \qquad (2.13)$$

Эти коэффициенты **x** и **y** называются коэффициентами Безу.

Обычный алгоритм Евклида основан на том, что $\text{НОД}\,(\boldsymbol{a},\ \boldsymbol{b}) = \text{НОД}\,(\boldsymbol{b},\ \boldsymbol{a}\ mod\ \boldsymbol{b})$.

Расширенный алгоритм параллельно отслеживает, как текущий остаток выражается через исходные $\boldsymbol{a}$ и $\boldsymbol{b}$.

Если мы знаем, что:

$$\boldsymbol{r}_{\text{i}} = \boldsymbol{a} \cdot \boldsymbol{x}_{\text{i}} + \boldsymbol{b} \cdot \boldsymbol{y}_{\text{i}}\,,$$

то на каждом шаге деления с остатком мы можем пересчитать коэффициенты.



Теорема 2.9. *Пусть a и b — два целых положительных числа. Тогда существуют целые (не обязательно положительные) числа x и y, такие, что*

$$ax + by = \gcd(a, b). \tag{2.13}$$

Обобщенный алгоритм Евклида служит для отыскания $\gcd(a, b)$ и x, y, удовлетворяющих (2.13). Введем три строки $U = (u_1, u_2, u_3)$, $V = (v_1, v_2, v_3)$ и $T = (t_1, t_2, t_3)$. Тогда алгоритм записывается следующим образом.

Алгоритм 2.2. Обобщенный алгоритм Евклида

ВХОД: Положительные целые числа a, b, $a \geq b$.
ВЫХОД: $\gcd(a,b)$, x, y, удовлетворяющие (2.13).

1. $U \leftarrow (a, 1, 0)$, $V \leftarrow (b, 0, 1)$.
2. WHILE $v_1 \neq 0$ DO
3. $q \leftarrow u_1 \text{ div } v_1$;
4. $T \leftarrow (u_1 \bmod v_1, u_2 - qv_2, u_3 - qv_3)$;
5. $U \leftarrow V$, $V \leftarrow T$.
6. RETURN $U = (\gcd(a,b), x, y)$.

Результат содержится в строке U.

Операция div в алгоритме — это целочисленное деление

$$a \text{ div } b = \lfloor a/b \rfloor.$$

Пример 2.12. Пусть a=28, b=19. Найдем числа x и y, удовлетворяющие (2.13).

U				28	1	0	
V	U			19	0	1	
T	V	U		9	1	-1	$q = 1$
	T	V	U	1	-2	3	$q = 2$
		T	V	0	19	-28	$q = 9$

Сложность алгоритма

Время:

O (log min(a, b)) — как и у обычного алгоритма Евклида.

Память:

O (1) для итеративной версии, **O(log n)** для рекурсивной (из-за стека).

Связь с обратным элементом по модулю

Если НОД $(a, m) = 1$, то из соотношения Безу:

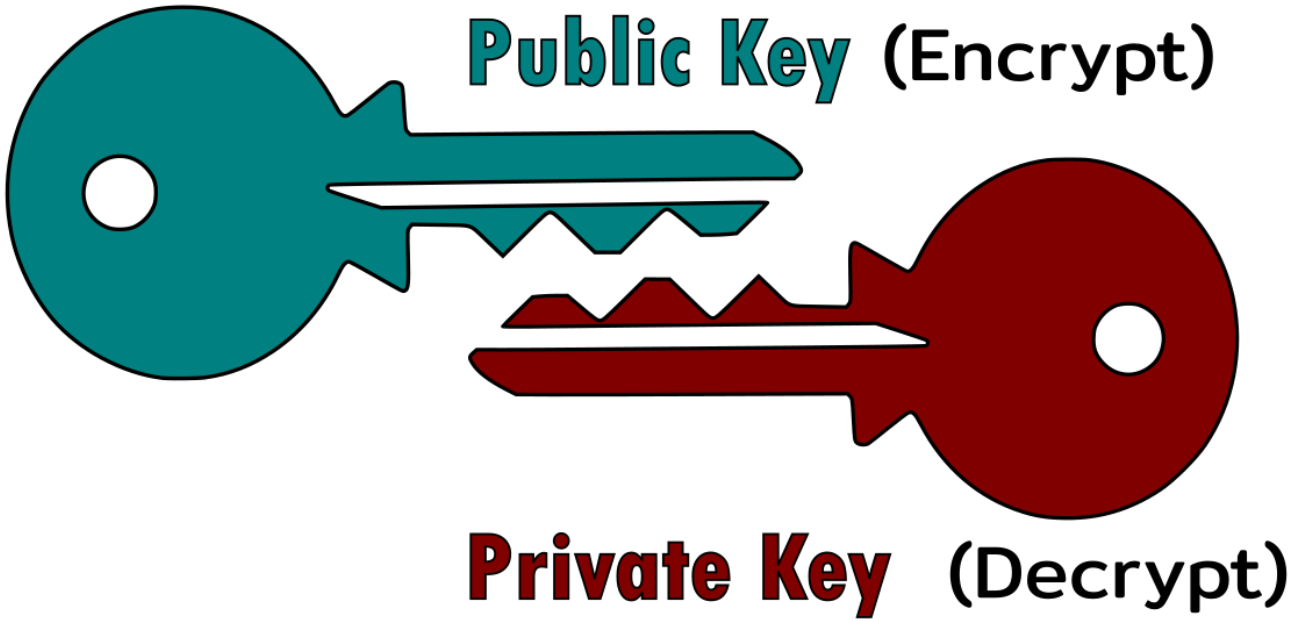
$$\boldsymbol{a \cdot x + m \cdot y = 1}$$

следует, что $a \cdot x \equiv 1 \, (mod \, m)$.

Значит, $\boldsymbol{x}$ — это обратный элемент к $\boldsymbol{a}$ по модулю $\boldsymbol{m}$.

Зачем это нужно в криптографии?

Нахождение обратного элемента по модулю — ключевая операция в криптографии (шифры Шамира, RSA и другие) в модульной арифметике.



Инверсия

Рассмотрим одно важное применение обобщенного алгоритма Евклида. Во многих задачах криптографии для заданных чисел c, m требуется находить такое число $d < m$, что

$$cd \bmod m = 1. \tag{2.14}$$

Отметим, что такое d существует тогда и только тогда, когда числа c и m взаимно простые.

Определение 2.6. Число d, удовлетворяющее (2.14), называется *инверсией c по модулю m* и часто обозначается $c^{-1} \bmod m$.

Данное обозначение для инверсии довольно естественно, так как мы можем теперь переписать (2.14) в виде

$$cc^{-1} \bmod m = 1.$$

Умножение на c^{-1} соответствует делению на c при вычислениях по модулю m. По аналогии можно ввести произвольные отрицательные степени при вычислениях по модулю m:

$$c^{-e} = (c^e)^{-1} = (c^{-1})^e \pmod{m}.$$

Пример 2.13. $3 \cdot 4 \bmod 11 = 1$, поэтому число 4 — это инверсия числа 3 по модулю 11. Можно записать $3^{-1} \bmod 11 = 4$. Число $5^{-2} \bmod 11$ может быть найдено двумя способами:

$$5^{-2} \bmod 11 = (5^2 \bmod 11)^{-1} \bmod 11 = 3^{-1} \bmod 11 = 4,$$

$$5^{-2} \bmod 11 = (5^{-1} \bmod 11)^2 \bmod 11 = 9^2 \bmod 11 = 4.$$

При вычислениях по второму способу мы использовали равенство $5^{-1} \bmod 11 = 9$. Действительно, $5 \cdot 9 \bmod 11 = 45 \bmod 11 = 1$.

$$cd \bmod m = 1. \tag{2.14}$$

Покажем, как можно вычислить инверсию с помощью обобщенного алгоритма Евклида. Равенство (2.14) означает, что для некоторого целого k

$$cd - km = 1. \tag{2.15}$$

Учитывая, что c и m взаимно просты, перепишем (2.15) в виде

$$m(-k) + cd = \gcd(m, c), \tag{2.16}$$

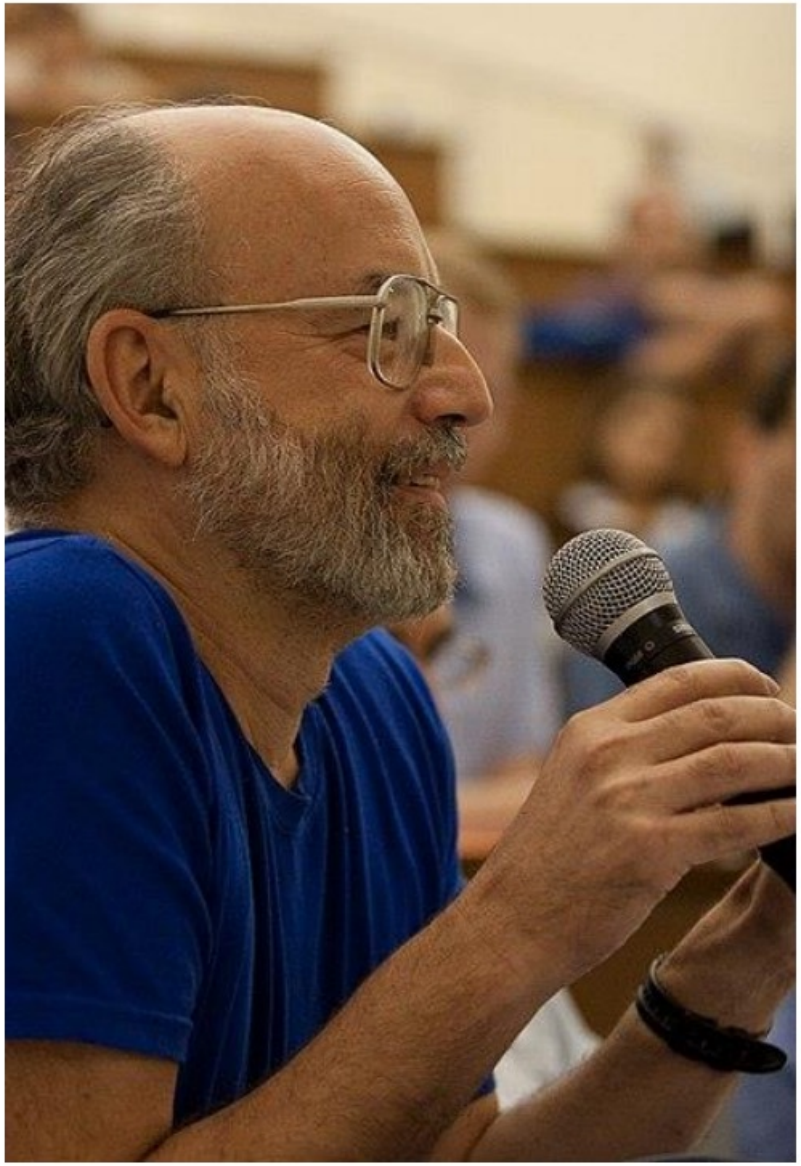
$$ax + by = \gcd(a, b).$$

Пример 2.14. Вычислим $7^{-1} \bmod 11$. Используем такую же схему записи вычислений, как в примере 2.12 :

$$\begin{array}{rrl} 11 & 0 & \\ 7 & 1 & \\ 4 & -1 & q = 1 \\ 3 & 2 & q = 1 \\ 1 & -3 & q = 1 \\ 0 & 11 & q = 3. \end{array}$$

Получаем $d = -3$ и $d \bmod 11 = 11 - 3 = 8$, т.е. $7^{-1} \bmod 11 = 8$.
Проверим результат: $7 \cdot 8 \bmod 11 = 56 \bmod 11 = 1$.

Шифр Шамира



~ 1980 г.

Ади Шамир (Adi Shamir)

описал трехэтапный протокол обмена зашифрованными сообщениями.

Описание алгоритма

Перейдем к описанию системы. Пусть есть два абонента A и B, соединенные линией связи. A хочет передать сообщение m абоненту B так, чтобы никто не узнал его содержание. A выбирает случайное большое простое число p и открыто передает его B. Затем A выбирает два числа c_A и d_A, такие, что

$$c_A d_A \bmod (p-1) = 1. \tag{2.17}$$

Эти числа A держит в секрете и передавать не будет. B тоже выбирает два числа c_B и d_B, такие, что

$$c_B d_B \bmod (p-1) = 1, \tag{2.18}$$

и держит их в секрете.

Формирование секретных ключей C_A , D_A на стороне А

$$C_A D_A \, mod \, (p-1) = 1$$

C_A – генерируется случайное число в диапазоне $(2;\ p-1)$, такое, что НОД $(C_A\ , p-1)\ = 1$

D_A можно найти с помощью обобщенного алгоритма Евклида.

Входные параметры: $p-1$, C_A

Тогда выходными параметрами будут: 1, $-k$, D_A

Формирование секретных ключей C_B , D_B на стороне B (точно так же, как на стороне A)

$$C_B D_B \, mod \, (p-1) = 1$$

C_B – генерируется случайное число в диапазоне $(2;\ p-1)$, такое, что НОД $(C_B\ , p-1)\ = 1$

D_B можно найти с помощью обобщенного алгоритма Евклида.

Входные параметры: $p-1$, C_B

Тогда выходными параметрами будут: 1, $-k$, D_B

После этого A передает свое сообщение m, используя трехступенчатый протокол. Если $m < p$ (m рассматривается как число), то сообщение m передается сразу , если же $m \geq p$, то сообщение предствляется в виде $m_1, m_2, \ldots, m_t$, где все $m_i < p$, и затем передаются последовательно $m_1, m_2, \ldots, m_t$. При этом для кодирования каждого m_i лучше выбирать случайно новые пары (c_A, d_A) и (c_B, d_B) — в противном случае надежность системы понижается.

В настоящее время такой шифр, как правило, используется для передачи чисел, например, секретных ключей, значения которых меньше p. Таким образом, мы будем рассматривать только случай $m < p$. Дадим описание протокола.

Шаг 1. A вычисляет число

$$x_1 = m^{c_A} \bmod p, \tag{2.19}$$

где m — исходное сообщение, и пересылает x_1 к B.

Шаг 2. B, получив x_1, вычисляет число

$$x_2 = x_1^{c_B} \bmod p \tag{2.20}$$

и передает x_2 к A.

Шаг 3. A вычисляет число

$$x_3 = x_2^{d_A} \bmod p \tag{2.21}$$

и передает его B.

Шаг 4. B, получив x_3, вычисляет число

$$x_4 = x_3^{d_B} \bmod p. \tag{2.22}$$

Утверждение 2.10 (свойства протокола Шамира).

1) *$x_4 = m$, т.е. в результате реализации протокола от A к B действительно передается исходное сообщение;*

2) *злоумышленник не может узнать, какое сообщение было передано.*

Доказательство. Вначале заметим, что любое целое число $e \geq 0$ может быть представлено в виде $e = k(p-1) + r$, где $r = e \bmod (p-1)$. Поэтому на основании теоремы Ферма

$$x^e \bmod p = x^{k(p-1)+r} \bmod p = \\ = (1^k \cdot x^r) \bmod p = x^{e \bmod (p-1)} \bmod p. \quad (2.23)$$

Справедливость первого пункта утверждения вытекает из следующей цепочки равенств:

$$\begin{aligned} x_4 &= x_3^{d_B} \bmod p = (x_2^{d_A})^{d_B} \bmod p = \\ &= (x_1^{c_B})^{d_A d_B} \bmod p = (m^{c_A})^{c_B d_A d_B} \bmod p = \\ &= m^{c_A d_A c_B d_B} \bmod p = m^{(c_A d_A c_B d_B) \bmod (p-1)} \bmod p = m \end{aligned}$$

(предпоследнее равенство следует из (2.23), а последнее выполняется в силу (2.17) и (2.18)).

Доказательство второго пункта утверждения основано на предположении, что для злоумышленника, пытающегося определить m, не существует стратегии более эффективной, чем следующая. Вначале он вычисляет c_B из (2.20), затем находит d_B и, наконец, вычисляет $x_4 = m$ по (2.22). Но для осуществления этой стратегии злоумышленник должен решить задачу дискретного логарифмирования (2.20), что практически невозможно при больших p.

Шаг 1. A вычисляет число

$$x_1 = m^{c_A} \bmod p, \tag{2.19}$$

Шаг 2. B, получив x_1, вычисляет число

$$x_2 = x_1^{c_B} \bmod p \tag{2.20}$$

Шаг 3. A вычисляет число

$$x_3 = x_2^{d_A} \bmod p \tag{2.21}$$

Шаг 4. B, получив x_3, вычисляет число

$$x_4 = x_3^{d_B} \bmod p. \tag{2.22}$$

Пример 2.15. Пусть A хочет передать B сообщение $m = 10$. A выбирает $p = 23$, $c_A = 7$ $(\gcd(7, 22) = 1)$ и вычисляет $d_A = 19$. Аналогично, B выбирает параметры $c_B = 5$ (взаимно простое с 22) и $d_B = 9$. Переходим к протоколу Шамира.

Шаг 1. $x_1 = 10^7 \bmod 23 = 14$.

Шаг 2. $x_2 = 14^5 \bmod 23 = 15$.

Шаг 3. $x_3 = 15^{19} \bmod 23 = 19$.

Шаг 4. $x_4 = 19^9 \bmod 23 = 10$.

Таким образом, B получил передаваемое сообщение $m = 10$.